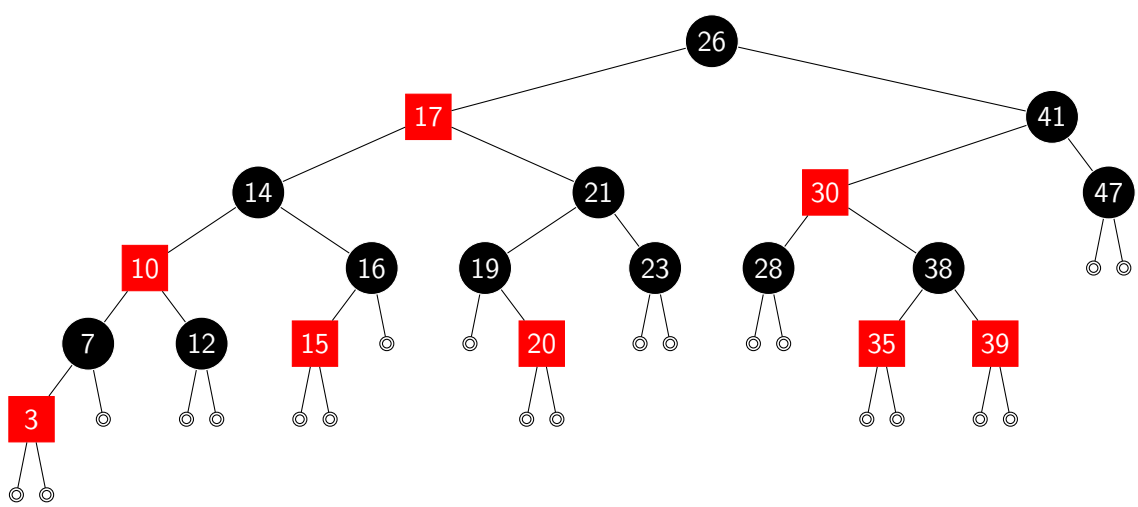
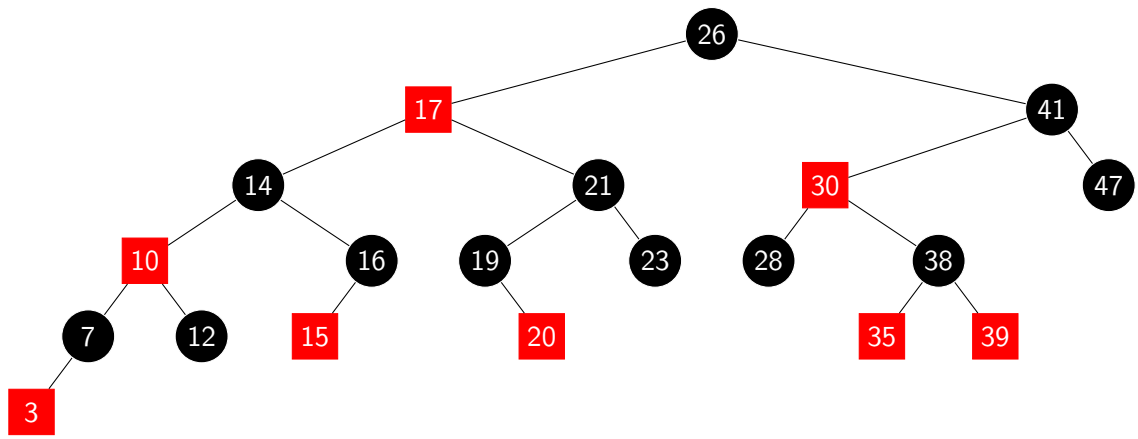
Introduction

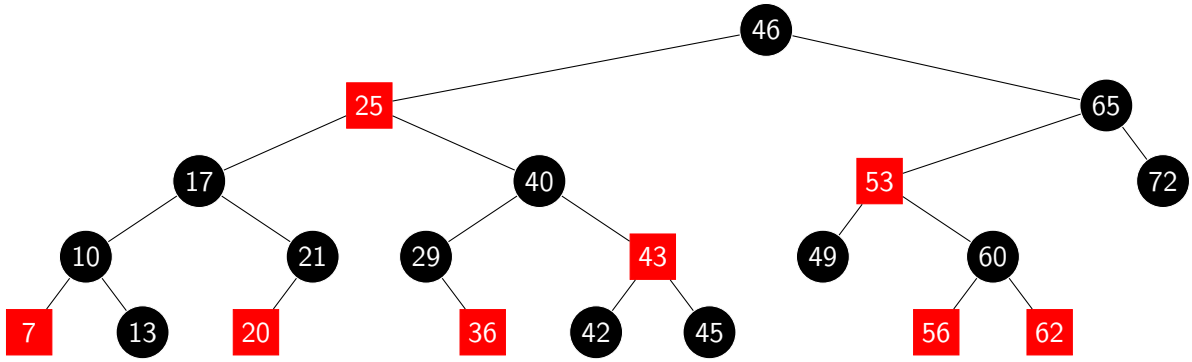
Definition 1. A **red-black tree** is a binary search tree with the following properties

1. Every node is either **red** or **black**.
2. The root is **black**.
3. Every leaf is NIL and is **black**.
4. If a node is **red**, then both its children are **black**.
5. For each node, all simple paths from the node to descendant leaves contain the same number of **black** nodes.

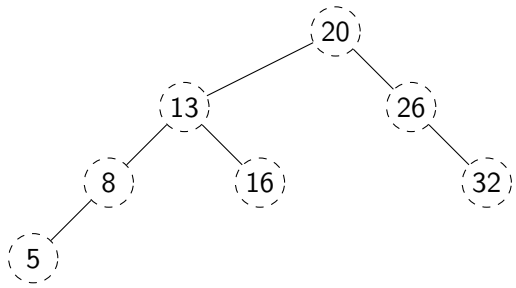




This tree is not a **red-black** tree. Why not?



Color the following tree so that it is a **red-black** tree



Theorems and correctness

Let h be the height of some **red-black** tree and x be a leaf of the **red-black** tree. How many nodes can be on a path from the root to x?

Theorem 1 (**Red-black** tree height)**.** A **red-black** tree with n internal nodes has height at most $2\log_2(n+1)$,

Theorem 2. A complete binary search tree of height h has $2^{h+1}-1$ nodes.

Proof. The ith level has 2^i nodes, where $i = 0, 1, \ldots, h$. Therefore the total number of nodes is

$$n = 1 + 2 + 2^2 + 2^3 + \cdots + 2^h = 2^{h+1} - 1$$

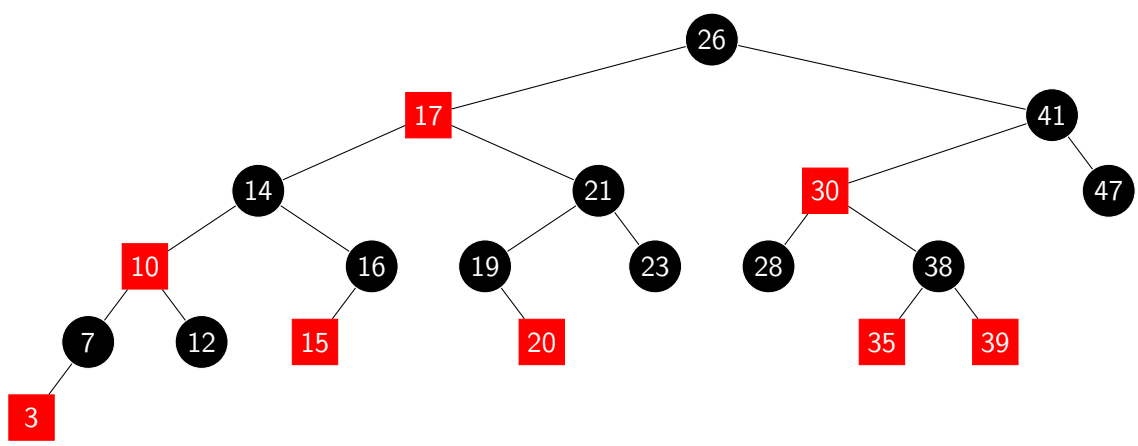
Lastly note that

$$\log_2(n+1) = 2^{h+1} \qquad \Longrightarrow \qquad h = \log_2(n+1) - 1$$

$\square$

Theorem 3. A **red-black** tree of height h has at least $2^{\lceil h/2 \rceil} - 1$ internal nodes.

Proof. Let T be a **red-black** tree of height h. Let T' be the tree formed by removing all of the leaves from T. Note that the height of T' must be $h-1$. Let r be the root of T'. Since no child of a **red** node is **red** and r is **black**, the longest path from r to a leaf has at least $\lceil h/2 \rceil$ **black** nodes (see Lemma 1 at the end of the notes for more details). Since every path from r to a leaf has the same number of **black** nodes, every path from r to a leaf at least $\lceil h/2 \rceil$ **black** nodes. Thus, T' contains a complete binary tree of height at least $\lceil h/2 \rceil - 1$. By Theorem 2, tree T' has at least $2^{(\lceil h/2 \rceil - 1)+1} - 1$ nodes. Therefore, T has at least $2^{\lceil h/2 \rceil} - 1$ internal nodes. $\square$



Theorem 1. A **red-black** tree with n internal nodes has height at most $2\log_2(n+1)$.

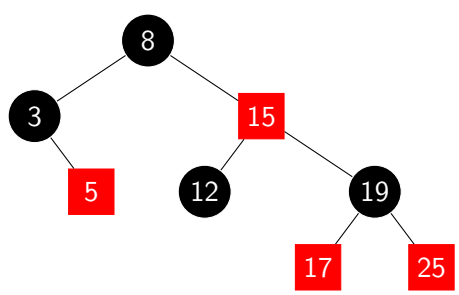
Proof. Let T be a **red-black** tree with n nodes and height h. By Theorem 3, $n \geq 2^{\lceil h/2 \rceil} - 1$. Therefore,

$$\log_2(n+1) \geq \lceil h/2 \rceil \geq h/2 \qquad \Longrightarrow \qquad h \leq 2\log_2(n+1)$$

$\square$

Methods and analysis

```
function RBLocateParent(T,z)
    ▷ Return future parent of z in tree T
    y = NIL
    x = T.root
    while x is not a leaf do
        y = x
        if z.key < x.key then
            x = x.left
        else
            x = x.right
        end if
    end while
    return y
end function
```

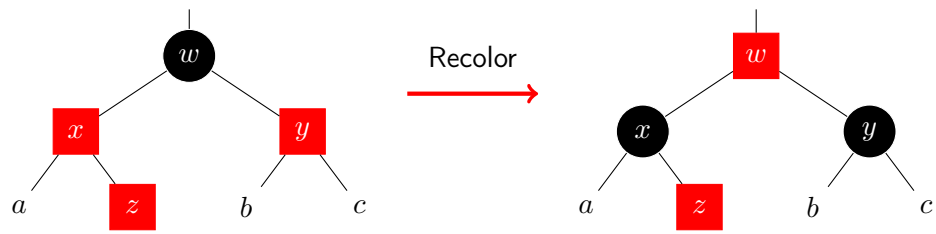



```
function RBTreeInsert(T,z)
    y = RBLocateParent(T, z)
    z.parent = y
    if y = NIL then
        T.root = z
    else if z.key < y.key then
        y.left = z
    else
        y.right = z
    end if
    z.left = leaf
    z.right = leaf
    z.color = red
    RBInsertFixup(T,z)
end function
```

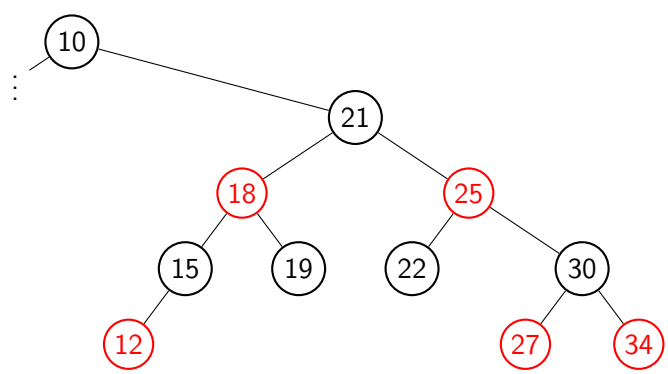
What are the possibilities that need fixing?

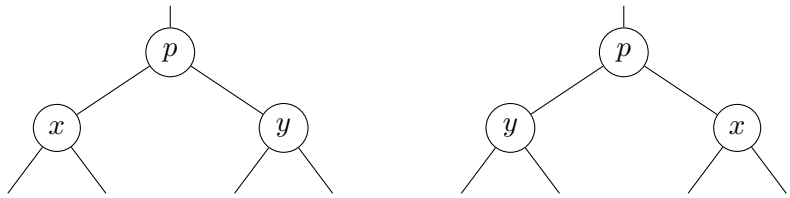
Case I If the parent and "auncle" of z are **red**

- Color the parent z **black**.
- Color the auncle of z **black**.
- Color the grandparent of z **red**.
- Repeat on the grandparent of z.

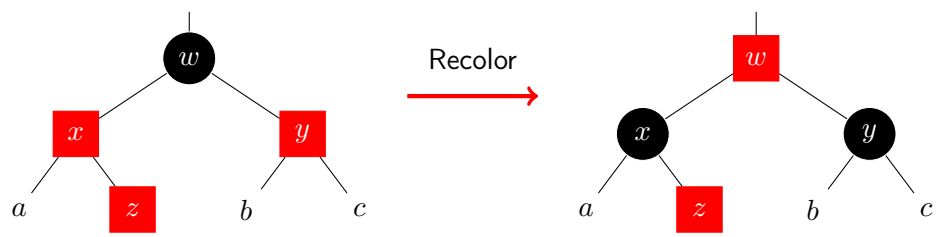


Perform RBTreeInsert(T,29) on the tree given below





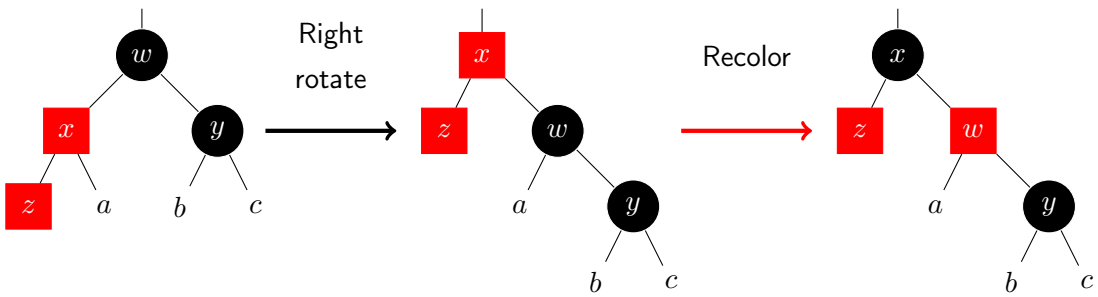

```
function Sibling(x)
    ▷ Returns the sibling of x
    if x.parent = NIL then
        error "Root has no siblings."
    end if
    p = x.parent
    if p.left = x then
        return p.right
    else
        return p.left
    end if
end function
```



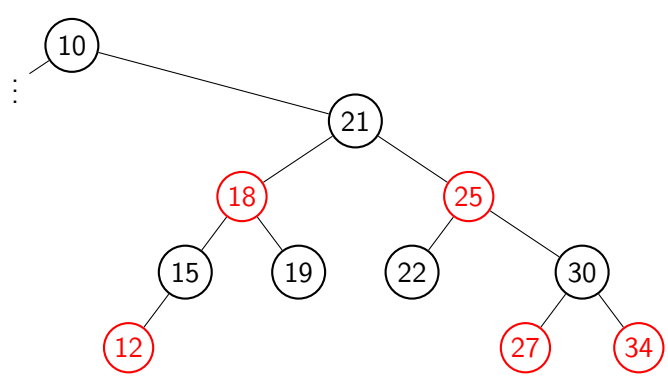

```
function RBInsertFixupA(T, alters z)
    while (z ≠ T.root) and (z.parent.color ≠ black) do
        y = Sibling(z.parent)
        if y.color = black then
            return
        end if
        z.parent.color = black
        y.color = black
        z = z.parent.parent
        z.color = red
    end while
end function
```

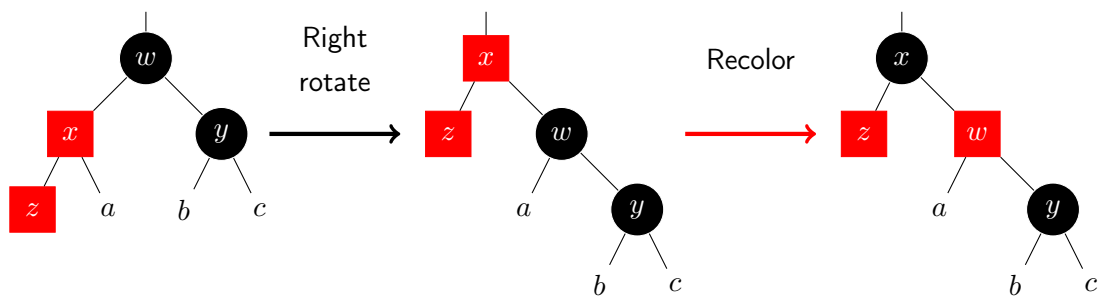
Case III If the parent of z is **red** and its auncle is **black**. If z is a left child and its parent is a left child,

- Right Rotate on the grandparent of z,
- Color the parent of z **black**,
- Color the sibling of z **red**



Perform `RBTreeInsert`(T,11) on the tree given below

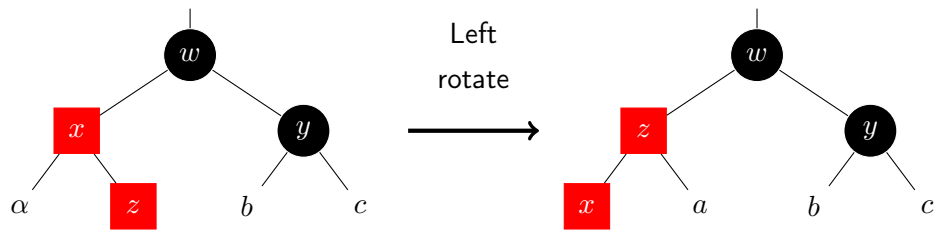




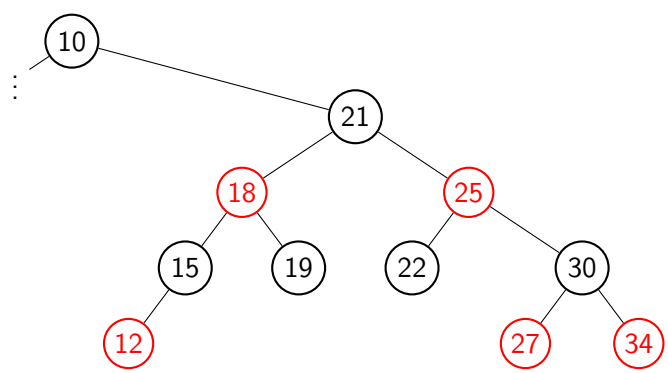
function `RBInsertFixupC`(T, **alters** z)
  if ($z = T.$`root`) **or** ($z.$`parent.color` $=$ **black**) **then**
    return
  end if
  $x = z.$`parent`
  $w = x.$`parent`
  if ($z = x.$`left`) **and** ($x = w.$`left`) **then**
    `RightRotate`(T,w)
    $x.$`color` $=$ **black**
    $w.$`color` $=$ **red**
  else if ($z = x.$`right`) **and** ($x = w.$`right`) **then**
    `LeftRotate`(T,w)
    $x.$`color` $=$ **black**
    $w.$`color` $=$ **red**
  end if
end function

Case II If the parent x of z is **red** and its auncle is **black**. If z is a right child and its parent x is a left child,

- Left Rotate on x,
- Apply algorithm from Case III on x

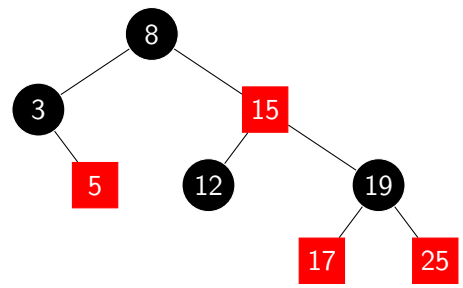


Perform `RBTreeInsert`(T,13) on the tree given below

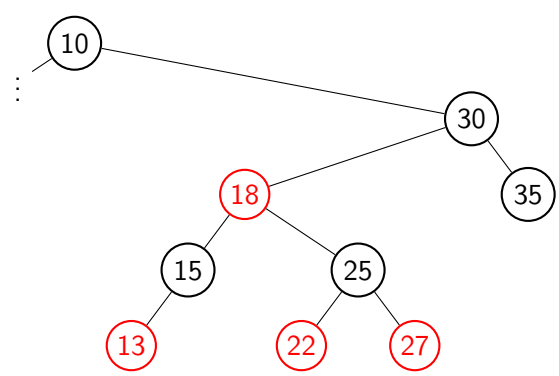


```
function RBInsertFixupB(T, alters z)
    if (z = T.root) or (z.parent.color = black) then
        return
    end if
    x = z.parent
    w = x.parent
    if (z = x.right) and (x = w.left) then
        z = x
        LeftRotate(T,x)
    else if (z = x.left) and (x = w.right) then
        z = x
        RightRotate(T,x)
    end if
end function
```

```
function RBInsertFixup(T,z)
    RBInsertFixupA(T,z)
    RBInsertFixupB(T,z)
    RBInsertFixupC(T,z)
    T.root.color = black
end function
```



Perform the RBTreeInsert(T,28) on the tree given below.

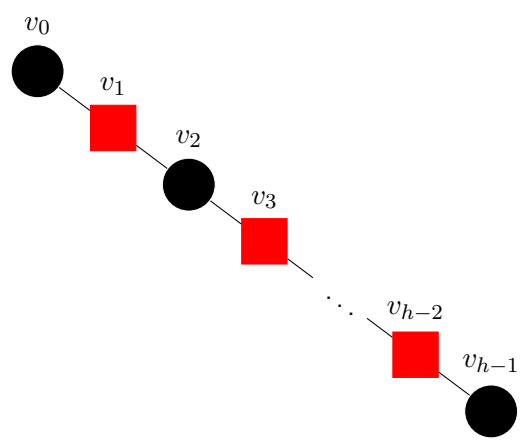


Appendix

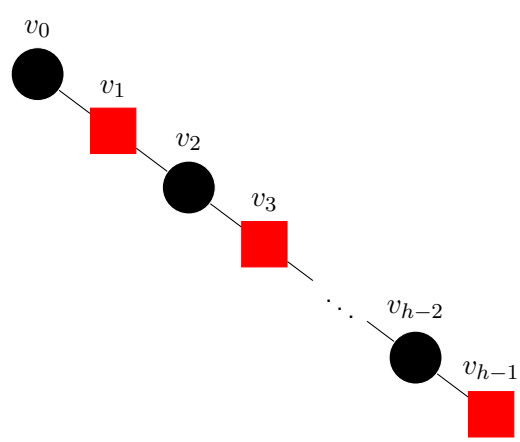
Lemma 1. Suppose T is a "trimmed" **red-black** tree (one where we have removed the **black NIL** nodes) of height $h-1$. The maximum number of **red** nodes in any simple path of length $h-1$ from the root to a leaf is $\lfloor h/2 \rfloor$ and the **minimum** number of black nodes is $\lceil h/2 \rceil$.

Proof. Let $p = \{v_0, v_1, v_2, \ldots, v_{h-1}\}$ be a simple path of length $h-1$ from the root of the tree to a leaf. Further suppose that p is such that it has the maximum number of **red** nodes possible. Notice that we cannot have subsequent nodes both **red**, so the maximum number of red nodes possible is obtained when nodes alternate between **black** and **red**. We continue by considering the parity of $h-1$;

If $h-1$ is even: Then nodes $v_0, v_2, v_4, \ldots, v_{h-1}$ are all **black** (since the root is always **black**) and nodes $v_1, v_3, v_5, \ldots, v_{h-2}$ are all **red**. That is, there are $\frac{h-1}{2}$ **red** nodes and $\frac{h+1}{2}$ **black** nodes.



If $h-1$ is odd: Then nodes $v_0, v_2, v_4, \ldots, v_{h-2}$ are all **black** (since the root is always **black**) and nodes $v_1, v_3, v_5, \ldots, v_{h-1}$ are all **red**. That is, there are $\frac{h}{2}$ **red** nodes and $\frac{h}{2}$ **black** nodes.



Now, we observe that

$$\lfloor h/2 \rfloor = \begin{cases} \frac{h-1}{2} & \text{if } h-1 \text{ is even} \\ \frac{h}{2} & \text{if } h-1 \text{ is odd} \end{cases} \tag{1}$$

and

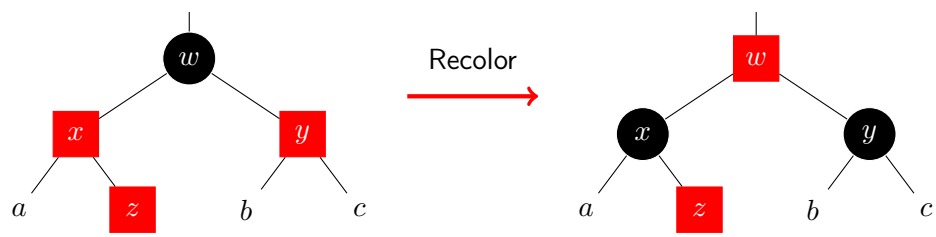
$$\lceil h/2 \rceil = \begin{cases} \frac{h+1}{2} & \text{if } h-1 \text{ is even} \\ \frac{h}{2} & \text{if } h-1 \text{ is odd} \end{cases}. \tag{2}$$

Therefore, the maximum number of **red** nodes in any simple path from the root to a leaf of length $h-1$ is $\lfloor h/2 \rfloor$ and the minimum number of **black** nodes is $\lceil h/2 \rceil$. $\square$

Summary of Cases

Case I If the parent and "auncle" of z are **red**

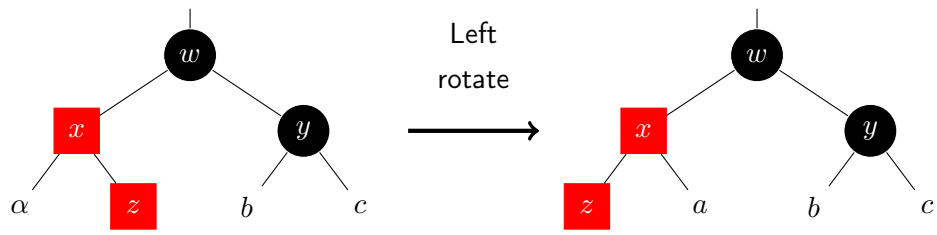
- Color the parent z **black**.
- Color the auncle of z **black**.
- Color the grandparent of z **red**.
- Repeat on the grandparent of z.



function `RBInsertFixupA`(T, **alters** z)
  while ($z \neq T$.`root`) **and** (z.`parent.color` $\neq$ **black**) **do**
    $y =$ `Sibling`(z.`parent`)
    if y.`color` $=$ **black then**
      return
    end if
    z.`parent.color` $=$ **black**
    y.`color` $=$ **black**
    $z = z$.`parent.parent`
    z.`color` $=$ **red**
  end while
end function

Case II If the parent x of z is **red** and its auncle is **black**. If z is a right child and its parent x is a left child,

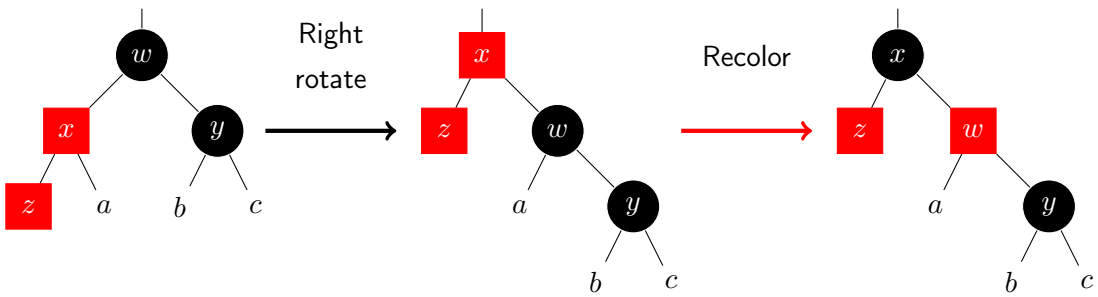
- Left Rotate on x,
- Swap z and x
- Apply algorithm from Case III on z




```
function RBInsertFixupB(T, alters z)
    if (z = T.root) or (z.parent.color = black) then
        return
    end if
    x = z.parent
    w = x.parent
    if (z = x.right) and (x = w.left) then
        z = x
        LeftRotate(T,x)
    else if (z = x.left) and (x = w.right) then
        z = x
        RightRotate(T,x)
    end if
end function
```

Case III If the parent of z is **red** and its auncle is **black**. If z is a left child and its parent is a left child,

- Right Rotate on the grandparent of z,
- Color the parent of z **black**,
- Color the sibling of z **red**




```
function RBInsertFixupC(T, alters z)
    if (z = T.root) or (z.parent.color = black) then
        return
    end if
    x = z.parent
    w = x.parent
    if (z = x.left) and (x = w.left) then
        RightRotate(T,w)
        x.color = black
        w.color = red
    else if (z = x.right) and (x = w.right) then
        LeftRotate(T,w)
        x.color = black
        w.color = red
    end if
end function
```